

Fundamentos de Programación (IAD-2413)

Instituto Tecnológico de Durango — Ingeniería en Inteligencia Artificial

Unidad 1: Introducción a la programación y conceptos fundamentales

Ejemplos trabajados en Python, C# y Java

Índice

- 1. Evolución de la programación
 - 2. Conceptos básicos: variables, tipos de datos, operadores
 - 3. Sintaxis en diferentes lenguajes de programación
 - 4. Tipado dinámico vs. tipado estático
 - 5. Uso del punto y coma
 - 6. Indentación en Python vs. llaves en C# y Java
 - 7. Estructuras de control: condicionales y bucles
 - 8. Definición de funciones y métodos
 - 9. Tabla resumen comparativa
 - 10. Conexión con la Práctica 1
-

1.1 Evolución de la programación

Generación	Época	Ejemplos	Características
Lenguaje máquina	1940s	Código binario	Directamente ejecutado por el hardware
Ensamblador	1950s	Assembly	Mnemotécnicos, requiere ensamblador
Alto nivel estructurado	1957-1970s	FORTRAN, COBOL, C	Sintaxis más legible, portabilidad
Orientado a objetos	1980s-1990s	C++, Java, C#	Clases, herencia, encapsulamiento

Generación	Época	Ejemplos	Características
Multiparadigma / actual	1990s-hoy	Python, JavaScript	Alta productividad, IA, ciencia de datos, web

Dato relevante para el curso: Java (1995) y C# (2000, Microsoft) comparten raíces sintácticas en C/C++, mientras que Python (1991) tomó un camino distinto priorizando la legibilidad sobre la estructura de bloques con llaves.

Actividad sugerida: línea del tiempo en equipos, destacando el año de aparición de Python, Java y C#, y el motivo por el cual cada uno se creó (Python: legibilidad y scripting; Java: “write once, run anywhere”; C#: ecosistema .NET de Microsoft).

1.2 Conceptos básicos: variables, tipos de datos, operadores

Variables

Espacio de memoria con nombre simbólico que almacena un valor modificable durante la ejecución del programa.

Tipos de datos básicos

Tipo	Python	C#	Java
Entero	int	int	int
Flotante	float	float / double	float / double
Carácter	(no existe, usa string de 1)	char	char
Cadena	str	string	String
Booleano	bool	bool	boolean

Operadores (comunes a los tres lenguajes)

- **Aritméticos:** + - * / % (Python además usa ** para potencia; C# y Java usan Math.Pow() / Math.pow())
- **Relacionales:** == != > < >= <=
- **Lógicos:** Python usa and or not; C# y Java usan && || !
- **De asignación:** = += -= *= /=

1.3 Sintaxis en diferentes lenguajes de programación

Declaración de variables

```
# Python
edad = 20
```

```
nombre = "Ana"
promedio = 8.5
es_regular = True
```

```
// C#
int edad = 20;
string nombre = "Ana";
double promedio = 8.5;
bool esRegular = true;
```

```
// Java
int edad = 20;
String nombre = "Ana";
double promedio = 8.5;
boolean esRegular = true;
```

Estructura mínima de un programa

```
# Python
def main():
    print("Hola, ITD")
```

```
main()
```

```
// C#
using System;

class Programa
{
    static void Main()
    {
        Console.WriteLine("Hola, ITD");
    }
}
```

```
// Java
public class Programa {
    public static void main(String[] args) {
        System.out.println("Hola, ITD");
    }
}
```

Tipado dinámico vs. tipado estático

Punto **conceptual clave** para entender por qué el mismo programa “se ve distinto” en cada lenguaje.

Aspecto	Tipado dinámico (Python)	Tipado estático (C# y Java)
---------	--------------------------	-----------------------------

Aspecto	Tipado dinámico (Python)	Tipado estático (C# y Java)
¿Cuándo se define el tipo?	En tiempo de ejecución, según el valor asignado	En tiempo de compilación, se declara explícitamente
¿Se puede cambiar el tipo de una variable?	Sí	No, sin conversión explícita
Detección de errores de tipo	En ejecución (más tarde)	En compilación (más temprano)
Ejemplo	x = 5 luego x = "hola" es válido	int x = 5; luego x = "hola"; genera error de compilación
Ventaja	Rapidez para prototipar, menos código	Mayor seguridad y detección temprana de errores
Desventaja	Errores de tipo pueden aparecer tarde	Requiere más código boilerplate

Python: tipado dinámico

x = 5 *# x es int*

x = "cinco" *# ahora x es str, esto es válido*

// C#: tipado estático

int x = 5;

x = "cinco"; *// ERROR de compilación: no se puede convertir string a int*

// Java: tipado estático

int x = 5;

x = "cinco"; *// ERROR de compilación*

Nota: C# también permite tipado dinámico opcional con la palabra clave var (inferencia de tipo) o dynamic, pero sigue siendo fuertemente tipado internamente; no es lo mismo que el dinamismo real de Python.

Uso del punto y coma (;)

Lenguaje	¿Requiere ;?	Función
Python	No	Los saltos de línea delimitan las instrucciones
C#	Sí, obligatorio	Marca el final de cada sentencia
Java	Sí, obligatorio	Marca el final de cada sentencia

Python - sin punto y coma

a = 5

b = 10

suma = a + b

print(suma)

```
// C# - punto y coma obligatorio
int a = 5;
int b = 10;
int suma = a + b;
Console.WriteLine(suma);
```

Si un estudiante olvida el ; en C# o Java, el compilador arroja un error. En Python, olvidar un ; no importa porque no se usa para terminar sentencias (aunque sí se puede usar opcionalmente para poner varias instrucciones en una línea, algo no recomendado por estilo).

Indentación en Python vs. llaves {} en C# y Java

Uno de los conceptos que más confunde a quienes vienen de C# o Java.

- **Python:** la indentación (sangría) **no es estética, es sintaxis obligatoria**. Define qué instrucciones pertenecen a un bloque (if, for, función, clase, etc.). Un error de indentación provoca un `IndentationError`.
- **C# y Java:** los bloques se delimitan con llaves { }. La indentación es solo una convención de legibilidad; el programa funciona igual esté bien o mal indentado (aunque es una mala práctica no indentar).

Python

```
if edad >= 18:
    print("Es mayor de edad")
    print("Puede votar")
else:
    print("Es menor de edad")
```

// C# - Las llaves delimitan el bloque, La indentación es solo estilo

```
if (edad >= 18)
{
    Console.WriteLine("Es mayor de edad");
    Console.WriteLine("Puede votar");
}
else
{
    Console.WriteLine("Es menor de edad");
}
```

// Java - mismo principio que C#

```
if (edad >= 18) {
    System.out.println("Es mayor de edad");
    System.out.println("Puede votar");
} else {
    System.out.println("Es menor de edad");
}
```

Consecuencia práctica: en Python, mezclar espacios y tabulaciones de forma inconsistente puede romper el programa; en C#/Java esto jamás causaría un error de sintaxis (solo se vería “feo”).

1.4 Estructuras de control: condicionales y bucles

Las estructuras de control determinan **qué instrucciones se ejecutan y cuántas veces**. Primero los condicionales (decidir) y después los bucles (repetir).

Condicional simple: if

Ejecuta un bloque **solo si** la condición es verdadera. Si es falsa, el programa continúa sin ejecutarlo.

```
# Python
temperatura = 35
if temperatura > 30:
    print("Hace calor")

// C# - La condición va entre paréntesis
int temperatura = 35;
if (temperatura > 30)
{
    Console.WriteLine("Hace calor");
}

// Java - La condición va entre paréntesis
int temperatura = 35;
if (temperatura > 30) {
    System.out.println("Hace calor");
}
```

Condicional doble: if - else

Elige entre **dos caminos**: uno si la condición es verdadera y otro si es falsa.

```
# Python
calificacion = 85
if calificacion >= 70:
    print("Aprobado")
else:
    print("Reprobado")

// C#
int calificacion = 85;
if (calificacion >= 70)
{
    Console.WriteLine("Aprobado");
}
```

```

else
{
    Console.WriteLine("Reprobado");
}

// Java
int calificacion = 85;
if (calificacion >= 70) {
    System.out.println("Aprobado");
} else {
    System.out.println("Reprobado");
}

```

Condicional múltiple: elif (Python) / else if (C# y Java)

Evalúa las condiciones **en orden** y ejecuta solo el primer bloque cuya condición sea verdadera; el else final atrapa todo lo demás.

```

# Python: elif
calificacion = 85
if calificacion >= 90:
    print("Excelente")
elif calificacion >= 80:
    print("Notable")
elif calificacion >= 70:
    print("Aprobado")
else:
    print("Reprobado")

// C#: else if
int calificacion = 85;
if (calificacion >= 90)
{
    Console.WriteLine("Excelente");
}
else if (calificacion >= 80)
{
    Console.WriteLine("Notable");
}
else if (calificacion >= 70)
{
    Console.WriteLine("Aprobado");
}
else
{
    Console.WriteLine("Reprobado");
}

// Java: else if
int calificacion = 85;
if (calificacion >= 90) {

```

```

        System.out.println("Excelente");
    } else if (calificacion >= 80) {
        System.out.println("Notable");
    } else if (calificacion >= 70) {
        System.out.println("Aprobado");
    } else {
        System.out.println("Reprobado");
    }
}

```

Salida con calificación = 85: Notable en los tres lenguajes. Aunque 85 también cumple ≥ 70 , ese bloque ya no se evalúa: **el orden de las condiciones importa.**

Condiciones compuestas: operadores lógicos

Operación	Python	C# y Java	Resultado
Y (ambas verdaderas)	and	&&	Verdadero solo si ambas lo son
O (al menos una)	or		Verdadero si alguna lo es
Negación	not	!	Invierte el valor

Python

```

edad = 20
tiene_ine = True
if edad >= 18 and tiene_ine:
    print("Puede votar")
if edad < 18 or not tiene_ine:
    print("No vota")

```

// C#

```

int edad = 20;
bool tieneIne = true;
if (edad >= 18 && tieneIne)
{
    Console.WriteLine("Puede votar");
}
if (edad < 18 || !tieneIne)
{
    Console.WriteLine("No vota");
}

```

// Java

```

int edad = 20;
boolean tieneIne = true;
if (edad >= 18 && tieneIne) {
    System.out.println("Puede votar");
}
if (edad < 18 || !tieneIne) {
    System.out.println("No vota");
}

```

Cortocircuito: en A and B, si A es falso, B ni se evalúa; en A or B, si A es verdadero, B ni se evalúa. Los tres lenguajes lo hacen.

Selección por valor: match (Python) / switch (C# y Java)

Cuando una variable se compara contra **varios valores exactos**, es más legible que una cadena larga de else if.

Python 3.10 o superior

```
dia = 2
match dia:
    case 1:
        nombre = "Lunes"
    case 2:
        nombre = "Martes"
    case _:
        nombre = "Otro día"
print(nombre)
```

// C#

```
int dia = 2;
string nombre;
switch (dia)
{
    case 1:
        nombre = "Lunes";
        break;
    case 2:
        nombre = "Martes";
        break;
    default:
        nombre = "Otro día";
        break;
}
Console.WriteLine(nombre);
```

// Java

```
int dia = 2;
String nombre;
switch (dia) {
    case 1:
        nombre = "Lunes";
        break;
    case 2:
        nombre = "Martes";
        break;
    default:
        nombre = "Otro día";
        break;
}
System.out.println(nombre);
```

- case _ (Python) y default (C# y Java) son el “en cualquier otro caso”.
- En C# y Java cada case termina con break. Python no lo necesita.
- Si a un case de **Java** se le olvida el break, la ejecución **continúa en el siguiente case** (*fall-through*). En **C#** esto es un error de compilación (CS0163).

Operador ternario: un if-else en una sola línea

Es una **expresión**: produce un valor que se puede asignar directamente a una variable.

```
# Python: valor_si_verdadero if condicion else valor_si_falso
estado = "Aprobado" if calificacion >= 70 else "Reprobado"
```

```
// C#: condicion ? valor_si_verdadero : valor_si_falso
string estado = calificacion >= 70 ? "Aprobado" : "Reprobado";
```

```
// Java: condicion ? valor_si_verdadero : valor_si_falso
String estado = calificacion >= 70 ? "Aprobado" : "Reprobado";
```

Errores comunes con condicionales

Error	Lenguaje	Qué ocurre
if x = 5 en lugar de if x == 5	Todos	= asigna, == compara. Python y Java marcan error; C# también.
Comparar cadenas con ==	Java	== compara referencias, no contenido. Usar cadena.equals("texto"). En C# y Python == sí compara contenido.
; justo después del if (...)	C# y Java	El if queda vacío y el bloque siguiente siempre se ejecuta (C# lo avisa con una advertencia).
Olvidar : al final de if / else	Python	SyntaxError.
Indentación incorrecta del bloque	Python	IndentationError, o el código queda fuera del if sin avisar.
Olvidar break en un case	Java / C#	Java: fall-through silencioso. C#: error CS0163.

Bucle for

Python

```
for i in range(5):  
    print(i)
```

// C#

```
for (int i = 0; i < 5; i++)  
{  
    Console.WriteLine(i);  
}
```

// Java

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

Bucle while

Python

```
contador = 0  
while contador < 5:  
    print(contador)  
    contador += 1
```

// C#

```
int contador = 0;  
while (contador < 5)  
{  
    Console.WriteLine(contador);  
    contador++;  
}
```

// Java

```
int contador = 0;  
while (contador < 5) {  
    System.out.println(contador);  
    contador++;  
}
```

1.5 Sintaxis para definición de funciones y métodos

Python: no requiere declarar tipo de retorno ni de parámetros

```
def sumar(a, b):  
    return a + b
```

```
print(sumar(3, 4))
```

// C#: método dentro de una clase, tipos explícitos

```
class Operaciones
```

```

{
    static int Sumar(int a, int b)
    {
        return a + b;
    }
}

// Java: método dentro de una clase, tipos explícitos
class Operaciones {
    static int sumar(int a, int b) {
        return a + b;
    }
}

```

Tabla resumen comparativa (para reforzar la unidad)

Característica	Python	C#	Java
Tipado	Dinámico	Estático (con var opcional)	Estático
Fin de sentencia	Salto de línea	;	;
Delimitación de bloques	Indentación	Llaves {}	Llaves {}
Condicional múltiple	if / elif / else	if / else if / else	if / else if / else
Operadores lógicos	and or not	&& \ \ !	&& \ \ !
Selección por valor	match / case (3.10+)	switch + break	switch + break
Ternario	a if c else b	c ? a : b	c ? a : b
¿Requiere clase para ejecutar?	No	Sí	Sí
Compilado / interpretado	Interpretado	Compilado a IL (CLR)	Compilado a bytecode (JVM)

Conexión con la Práctica 1 del temario

Con estos elementos, el equipo ya está listo para desarrollar la **Práctica 1: Calculadora básica**, implementando suma, resta, multiplicación y división con variables, operadores, condicionales, bucles y funciones en Python, C# y Java, comparando en su informe las diferencias de sintaxis abordadas en esta unidad (tipado, punto y coma, indentación/llaves).

*Material didáctico — Departamento de Sistemas y Computación, Instituto
Tecnológico de Durango (TecNM)*